

Pseudocode:

Problem Statement: Make an algorithm that tells the user the minimum amount of swaps needed to have correct pairs next to one another in a given array.

Input: an integer array

Output: # of swaps, new array (optional)

Constraints: Input array has to be even in size | Number of elements between 2 and 30 | Numbers given must have its correct pair | No duplicate numbers

```
Function numOfSwap (array, arraySize) {  
    int swapCounter;  
    for (first number and every other number in the array) {  
        if (their next number isn't their correct pair) {  
            for (current number to end of array) {  
                if (correct pair is found) {  
                    swap correct pair with current pair;  
                }  
                swapCounter++;  
            }  
        }  
    }  
}
```

Time Complexity

Initializing array & insertion: $1 + n$

Nested for-loop: n^2

Displaying User and Updated arrays: $2n$

Total: $1 + n + n^2 + 2n = n^2 + 3n + 1$

Proving Efficiency Class with Limits: $F(n) = n^2 + 3n + 1$, $g(n) = n^2$

$$\lim_{n \rightarrow \infty} \frac{n^2 + 3n + 1}{n^2} = \lim_{n \rightarrow \infty} 1 + \frac{3}{n} + \frac{1}{n^2} = 1 + \frac{3}{\infty} + \frac{1}{\infty} = 1$$

Therefore: $n^2 + 3n + 1$ is within $O(n^2)$

Space Complexity

Array: n

ints: 5

Total Space Complexity: $n + 5 = O(n)$